

Risposte domande di teoria

Qual è la definizione formale di linguaggio?

Il linguaggio è un insieme di simboli convenzionali, che si uniscono in enunciati più complessi in base a determinate regole.

Quali sono i 3 livelli di regole di un linguaggio?

- Sintattico: sono le regole che stabiliscono come si uniscono fra di loro i simboli per formare enunciati validi;
- Semantico: sono le regole che associano un significato agli enunciati;
- Pragmatico: sono le effettive conseguenze degli enunciati.

Come si associa la semantica alla grammatica?

La **semantica** si associa alla **grammatica** definendo il **significato** delle strutture sintattiche.

Esempio:

- **Grammatica:** `assegnamento → variabile = espressione;`
- **Semantica:** il significato dell'assegnamento è aggiornare il valore della variabile con il risultato dell'espressione.

Che cos'è Kleene Star?

E l'insieme di tutte le sottostringhe realizzabili a partire da un alfabeto sigma + le stringhe vuote.

Il linguaggio è un sottoinsieme di?

Kleene Star.

Definizione formale di automa a stati finiti?

E' una quintupla formata da:

- - K =tutti i possibili stati in cui si può trovare l'automa;
- - Σ =tutti i possibili segnali in ingresso;
- - δ =funzione di transizione di stato;
- - q_0 =stato iniziale;
- - F =stati finali.

Definizione macchina di Turing?

Macchina computazionale teorica basata su un nastro infinito diviso in celle che viene letto da una testina mobile.

Com'è composta la tabella di transizione di stato della MdT?

È formata da:

- - s=stato macchina istante presente;
- - i=simbolo letto su nastro istante presente;
- - S=stato futuro della macchina;
- - l=simbolo scritto nell'istante successivo;
- - V=direzione in cui si muoverà la testina.

Cos'è la macchina di Turing universale?

È una macchina di Turing che riceve in ingresso la codifica di un'altra macchina di Turing e la esegue.

Cos'è l'Abstract Syntax Tree?

È una rappresentazione grafica che evidenzia la struttura sintattica di una frase, ignorando grammatica e notazione.

Cos'è il Concrete syntax tree e come viene anche detto?

È un AST definito per una grammatica specifica. È detto anche Parse Tree.

Come si realizza un approccio generativo per identificare l'appartenenza di una parola ad un linguaggio formale?

Usando una grammatica formale.

Definizione formale di grammatica.

È una quadrupla formata da:

- - N =simboli non terminali;
- - Σ =simboli terminali/token;
- - R =insieme delle regole di produzione;
- - S =simboli non terminali speciali che precedono un enunciato valido.

Cos'è il Formato di Backus e Naur (BNF)?

Formalismo utilizzato in informatica per definire la corretta struttura sintattica di un enunciato.

Cos'è l'extended BNF e qual è la differenza con la BNF normale?

L'extended BNF è una BNF più espressiva.

In che caso una grammatica si dice ambigua?

Quando uno stesso enunciato produce più Parse Tree diversi.

Cos'è un'espressione regolare?

Sequenza di caratteri che definisce un pattern e che viene utilizzato per riconoscere stringhe di testo.

Cos'è una carta sintattica?

È una rappresentazione grafica che esprime le regole di una grammatica.

Definizione di algoritmo.

Insieme di istruzioni elementari e definite volte a risolvere una classe di problemi.

Quali sono le proprietà di un algoritmo?

- - Definito=le istruzioni non sono ambigue;
- - Finito=raggiunge un risultato;
- - Corretto=raggiunge un risultato corretto;
- - Efficiente=raggiunge un risultato nel minor numero di istruzioni possibile.

Cosa s'intende per complessità di un algoritmo?

Quante risorse impiega.

Cosa sono i diagrammi di flusso?

Sono diagrammi che mostrano il flusso di esecuzione di un programma.

Cos'è la programmazione non strutturata?

È la programmazione che fa uso di salti incondizionati.

Quali sono le caratteristiche di un modello di programmazione imperativo?

- - Dinamicità del valore delle variabili;
- - Staticità del programma.

Cos'è la programmazione strutturata?

È la programmazione in cui il flusso del programma è rigidamente regolato da costrutti di programmazione.

Cosa sono le variabili?

Sono porzioni di memoria, a cui viene associato un identificatore, e che servono a contenere un dato.

Cos'è un identificatore e quali rapporti ha con l'area di memoria e il suo valore?

L'identificatore è il nome che si dà ad una variabile. Con l'area di memoria ha un rapporto statico, mentre con il suo valore ha un rapporto dinamico.

Cos'è un costrutto?

Sono unità sintattiche che racchiudono istruzioni elementari per creare blocchi strutturati.

Quali sono i costrutti fondamentali?

- - Selezione: valuta una condizione e modifica il flusso del programma in base ad essa;
- - Iterazione (post/pre-condizionata): in base ad una condizione, ripete o meno un certo blocco di istruzioni;
- - Sequenza: le istruzioni vengono eseguite una dopo l'altra, in base all'ordine in cui sono state scritte.

Che tipi di iterazioni esistono e quali sono le differenze tra di loro?

- Pre-condizionata: vi è una selezione che valuta se il blocco istruzioni dell'iterazione vada eseguito o meno;
- Post-condizionata: vi è una selezione alla fine dell'iterazione che valuta se l'iterazione vada ripetuta o meno.

Come si definisce un programma proprio?

Un programma si definisce proprio quando ha un solo ingresso ed una sola uscita e quando ogni elemento appartiene ad un percorso che porta dall'ingresso all'uscita.

Quando due programmi si dicono equivalenti?

Quando, dati gli stessi elementi in ingresso, restituiscono gli stessi elementi in uscita.

Cosa dice il Teorema di Bohm Jacopini?

Il teorema di Bohm Jacopini afferma che un programma scritto in un linguaggio di programmazione non strutturata, può essere riscritto in un programma equivalente scritto in programmazione strutturata.

Cosa sono le trasformazioni di Ashcroft e Manna?

Sono semplificazioni che si applicano ai flow chart che consistono nel racchiudere parti di esso in sottoprogrammi propri.

Cos'è un'invariante?

È un'asserzione sempre vera.

Cos'è un'asserzione?

È una condizione che può essere vera o falsa.

Cosa si intende per visibilità di una variabile?

Si parla di ambito di visibilità quando il programma riconosce come definito l'identificatore della variabile.

Cos'è lo scope di una variabile?

È l'ambito di visibilità di una variabile.

Che relazione c'è fra i tipi di dato?

Una relazione gerarchica.

Cosa si intende per precedenza degli operatori?

S'intende la gerarchia tra gli operatori.

Cosa specifica il tipo di variabile (4)?

- - Tipo di valore;
- - Dimensione;
- - Codifica;
- - Operazioni.

Cos'è il casting?

È la conversione esplicita di una variabile da un tipo ad un altro tipo.

Come funziona l'assegnamento?

L'assegnamento è rappresentato dall'operatore '=' che ha come effetto primario il richiedere il valore della variabile/espressione alla sua destra, e, come effetto secondario, quello di copiarlo nella variabile alla sua sinistra.

Che differenza c'è fra If/else e un operatore ternario?

- If/Else modifica il flusso ma non produce valore;
- L'operatore ternario non modifica il flusso ma produce valore.

Quali sono i cicli iterativi in C?

- for;
- While;
- Do-While.

A cosa vale l'espressione $i=1,2,3; ?$

L'espressione vale 3. i vale 1.

Cos'è un vettore?

È un aggregatore di variabili omogenee tra loro, che vengono raccolte in oggetti identificati tramite un indice numerico.

Cos'è una struttura?

È un aggregatore di variabili non omogenee tra loro.

Cos'è il typedef?

È una definizione di tipo.

Quando si parla di matrici e vettori, che differenza c'è tra $V[i][j]$ e $V[i,j]$?

- $V[i][j]$ è la sintassi standard per accedere ad un elemento di una matrice;
- $V[i,j]$ equivale a $V[j]$ perché i viene scartata.

Come viene gestito in memoria un vettore?

In **C**, un **vettore** (o array) viene gestito in memoria come un blocco contiguo di celle di memoria.

Come si passa ad una funzione un vettore?

Passandogli il nome del vettore, che rappresenta un puntatore al primo elemento.

Come viene memorizzata in memoria una matrice?

In modo sequenziale riga per riga.

Cos'è una stringa e come si distingue da un vettore?

Una stringa è una sequenza di caratteri. Si distingue da un vettore per la presenza del carattere terminatore `'/0'`.

Che tipi di sottoprogrammi esistono?

- Procedure;
- Funzioni.

Cos'è lo stack?

È la parte di memoria che gestisce la chiamata ed il ritorno delle funzioni. Contiene il record di attivazione.

Cos'è il record di attivazione e cosa contiene?

È l'unità di base dello stack e contiene:

- Parametri delle funzioni;
- Indirizzi di ritorno;
- Puntatori ai record di attivazione precedenti;
- Variabili locali;
- Valori temporanei.

Nelle funzioni, che differenza c'è fra parametro formale e attuale?

- Il parametro formale è il nome del parametro inserito nel prototipo;
- Il parametro attuale è l'effettivo valore del parametro che viene fornito in chiamata.

Cos'è la tipizzazione di un linguaggio e in che gruppi si dividono i linguaggi tipizzati?

La tipizzazione sono le regole che un linguaggio impone sui tipi di dati.

I linguaggi fortemente tipizzati hanno regole stringenti sui tipi.

I linguaggi debolmente tipizzati hanno regole più permissive sui tipi.

Che modi ci sono di passare i parametri in C e non?

- Call by value (C)=i valori dei parametri attuali vengono copiati nei parametri formali;
- Call by reference (C)=nei parametri formali vengono inseriti dei puntatori ai parametri attuali;
- Call by result=copia i valore dei paramteri formali in quelli attuali in uscita;
- Call by value-result=copia il valore in chiamata e lo copia ancora in uscita dalla funzione;
- Call by name=passa il nome simbolico dei parametri consentendo accesso e modifica.

Cos'è un puntatore?

È una variabile che contiene un indirizzo di memoria.

Cos'è un Union?

È simile ad una struttura, ma il compilatore alloca lo spazio in memoria solo per il tipo più grosso.

Differenza tra `int **p` e `int *p[5]`?

- `int **p` dichiara un doppio puntatore;
- `int *p[5]` dichiara un array di puntatori.

Cosa si può usare quando una funzione deve restituire più di un solo valore?

- Call by reference;
- Strutture;
- Vettori dinamici.

In una matrice $v[2][3]$, cosa indica l'identificatore v ?

Identifica la prima riga della matrice.

Dato questo esempio:
Cosa stampa?

```
int arr[2][3]={
{11,22,33}
{44,55,66}
};
int (*p)[3];
p=arr;
printf("arr[%d][%d]=%d\n",i,j,*(*(p+i)+j));
```

- p è il nome della matrice;
- p+i indica che si sta avanzando di i righe;
- * è un puntatore che punta la prima casella della riga;
- +j significa che si avanza di j colonne;
- *serve a puntare la casella precisa.

Cosa fa la funzione malloc?

Permette di allocare dinamicamente memoria nello Heap run-time.

Cosa permette di fare l'allocazione di memoria dinamica?

Permette di creare vettori dinamici la cui dimensione può essere decisa run-time.

Cosa sono le liste concatenate?

Sono catene di vettori costituite da vettori che contengono i valori ed un puntatore al vettore seguente della catena.

Cos'è l'operatore ->?

Permette di accedere ai membri di una struttura tramite i puntatori.

Come è fatto il memory layout di un programma in C (6)?

- Segmento codice;
- Segmento dati;
- Segmento dati non inizializzati;
- Stack;
- Heap;
- Env e argomenti da riga di comando.

Cos'è la ricorsione?

È quando una funzione chiama se stessa.

Cos'è la ricorsione multipla?

È quando una funzione chiama più volte se stessa durante la sua esecuzione.

Cos'è winding e unwinding?

- Winding: catena di chiamate a funzione durante la ricorsione;
- Unwinding: catena di ritorni a funzione durante la ricorsione.

Cos'è uno stream?

È una qualsiasi fonte di dati in ingresso/una qualsiasi destinazione per i dati in uscita.

Che tipo di file esistono?

- Testo;
- Binario.

Cosa sono le macro e in cosa si distinguono dalle funzioni?

Le macro sono sostituzioni testuali effettuate dal pre-compilatore, che rimpiazza il nome della macro con il codice assegnato. Si distingue dalla funzione per:

- Mancanza di un codice e di variabili proprie;
- Non viene chiamata;
- Viene trattata dal pre-compilatore;
- Non effettua un controllo sugli argomenti.

A cosa equivale il nome della funzione?

Al suo indirizzo di memoria.

Che differenza c'è fra `int *f(int);` e `int (*f)(int);`?

- `int *f(int)` dichiaro una funzione che restituisce un puntatore;
- `int (*f)(int)` dichiaro un puntatore a funzione.

La struttura è un tipo di variabile?

No, è un aggregatore di variabili.

Cos'è const nel passaggio di parametri?

Impedisce alla funzione di modificare il valore puntato dal puntatore a cui è applicato.

Cos'è una variabile static?

È una variabile che mantiene il suo valore al di fuori del suo scope e viene condivisa dalle chiamate alla stessa funzione;

Cos'è una funzione static?

È una funzione il cui scope si limita al file in cui è definita.

Cos'è l'aritmetica dei puntatori?

Sono le operazioni aritmetiche che si possono applicare ai puntatori.

Cos'è il paradigma di programmazione e che tipi esistono?

Il paradigma di programmazione è un modello strutturale che i linguaggi di programmazione seguono per definire la loro struttura.

- Procedurale;
- Dichiarativo.

Quali sono le differenze tra programmazione imperativa e funzionale?

La programmazione imperativa segue il paradigma procedurale. È basata su istruzioni elementari fornite al calcolatore, sull'assegnamento e sulla modifica del valore delle variabili.

La programmazione funzionale segue il paradigma dichiarativo. I programmi spiegano al calcolatore cosa si vuole ottenere, ma non come lo si vuole ottenere. Non vi è l'assegnamento, ma i nuovi valori delle variabili vengono calcolati costruendo nuovi stati a partire da quelli precedenti.

Cos'è l'effetto collaterale di una funzione?

Si ha quando una funzione modifica un valore o uno stato al di fuori del proprio scope.

È una proprietà della programmazione imperativa.

Cos'è la trasparenza referenziale?

È quando un'espressione può essere sostituita dal suo valore.

Riferito ad una funzione, è una funzione che non utilizza e non modifica nulla al di fuori del suo scope.

È una proprietà della programmazione funzionale.

Cos'è la first-class citizen?

Un elemento è un first-class citizen quando può essere trattato come valori ordinari, ovvero:

- Assegnato ad una variabile;
- Passato ad una funzione;
- Restituito da una funzione.

Cos'è l'astrazione e che tipi di astrazione esistono (4)?

L'astrazione è quando si nascondono le implementazioni di un codice, mostrando solo ciò che è necessario per il suo utilizzo.

I vari tipi di astrazione sono:

- Procedurale;
- Modulare;
- Dei dati;
- Degli oggetti.

Che differenza c'è fra uno stile che un linguaggio supporta ed uno che permette?

Un linguaggio supporta uno stile di programmazione quando è pienamente ottimizzato per quello stile.

Un linguaggio permette uno stile di programmazione quando consente l'uso di quello stile, ma non è pienamente ottimizzato per esso.

Cos'è l'astrazione procedurale?

È l'astrazione di procedure e funzioni.

Cos'è l'astrazione modulare?

È l'astrazione di un modulo che contiene procedure e variabili.

Caratteristiche fondamentali dell'astrazione modulare?

- Ruolo;
- Interfaccia;
- Implementazione.

Cos'è l'incapsulation?

Raggruppamento per creare un'entità unica per protezione.

Cos'è la programmazione modulare?

È un tipo di programmazione i cui programmi sono costituiti da moduli che dialogano tra loro.

Cos'è l'astrazione dei dati?

È un modello che comprende un insieme associato di operazioni e un tipo.

Da cosa è definito il dominio dell'AdT?

Da assiomi e precondizioni che definiscono il contratto delle operazioni.

Che tipi di operatori sono definiti nell'astrazione dati (5)?

- Costruttore;
- Selettore;
- Trasformatore;
- Predicato;
- Funzione.

Quali sono le caratteristiche fondamentali dell'astrazione dati?

- Incapsulamento;
- Località dei cambiamenti;
- Flessibilità.

Cos'è il procedural data abstraction?

Quando si utilizzano funzioni per accedere a dei dati.

Cos'è l'astrazione ad oggetti?

L'**astrazione ad oggetti** è il principio della programmazione orientata agli oggetti che consente di modellare entità del mondo reale evidenziandone solo gli aspetti rilevanti, nascondendo i dettagli interni e rappresentandole tramite attributi e metodi.

Di cosa è dotato un oggetto?

- Identità;
- Stato;
- Comportamento.

A cosa reagisce un oggetto?

Ad una serie di messaggi che costituiscono la sua interfaccia.

Cos'è una classe di oggetti?

È un raggruppamento di oggetti accomunati da uno stesso concetto.

Cosa rappresenta l'oggetto per la classe?

Una sua istanza.

Cos'è un metodo?

È una funzione che va ad agire su un oggetto.

Che differenza c'è fra puntatori di C e riferimenti di Java?

I riferimenti di Java non hanno un'aritmetica dei puntatori come i puntatori in C.

Java che tipo di tipizzazione ha?

È fortemente tipizzato.

Che tipi ci sono in Java?

- Primitivi;
- Riferimento;
- Letterale null.

Che tipo di visibilità ci sono in Java?

- Private;
- Public;
- Non specificato;
- Protected.

Che differenza c'è fra attributi d'istanza e di classe?

Gli attributi d'istanza sono attributi definiti senza la keyword static.

Gli attributi di classe sono attributi definiti con la keyword static.

Cosa sono le stringhe?

Sono sequenze immutabili di caratteri.

Cosa sono le classi involucro?

Rappresentano i tipi primitivi.

Come viene applicata l'astrazione in Java?

- Suddivisione del codice in metodi;
- Organizzazione in file .java;
- L'uso di classi;
- La classe diventa oggetto tramite new.

Quali sono i principi SOLID?

- Principio di singola responsabilità: ogni elemento ha una sola responsabilità che incapsula.
- Principio aperto chiuso: ogni entità software dev'essere aperta alle espansioni ma chiusa alle modifiche;
- Principio di sostituibilità di Liskov: dato un programma che usa oggetti di una classe A, il funzionamento logico del programma non cambia se sostituisco gli oggetti della classe A con oggetti di una classe B;
- Principio di segregazione delle interfacce: un programma dovrebbe avere più interfacce specifiche, piuttosto che poche interfacce generiche;
- Principio di inversione delle dipendenze: è meglio che una classe dipenda da una classe astratta che da una concreta.

Cos'è una relazione di associazione?

Quando gli oggetti di una A possono interagire con gli oggetti di una classe B.

Cos'è una relazione di appartenenza?

Quando la funzionalità di un oggetto dipende dalla funzionalità combinata di più oggetti.

Cos'è una relazione di specializzazione?

È l'ereditarietà. Si ha quando una classe A può eseguire tutte le funzioni di una classe B, eventualmente estendendoli e/o modificandoli.

Cos'è un metodo astratto?

È un metodo privo d'implementazione.

Cos'è una classe astratta?

È una classe dotata sia di metodi concreti che di metodi astratti non istanziabile.

Cos'è un interfaccia?

È una classe con solo metodi astratti.

Che differenza c'è tra classe base e derivata?

La classe derivata è la specializzazione della classe base.

Cos'è l'ereditarietà d'interfaccia (tipi)?

Meccanismo di compatibilità fra tipi.

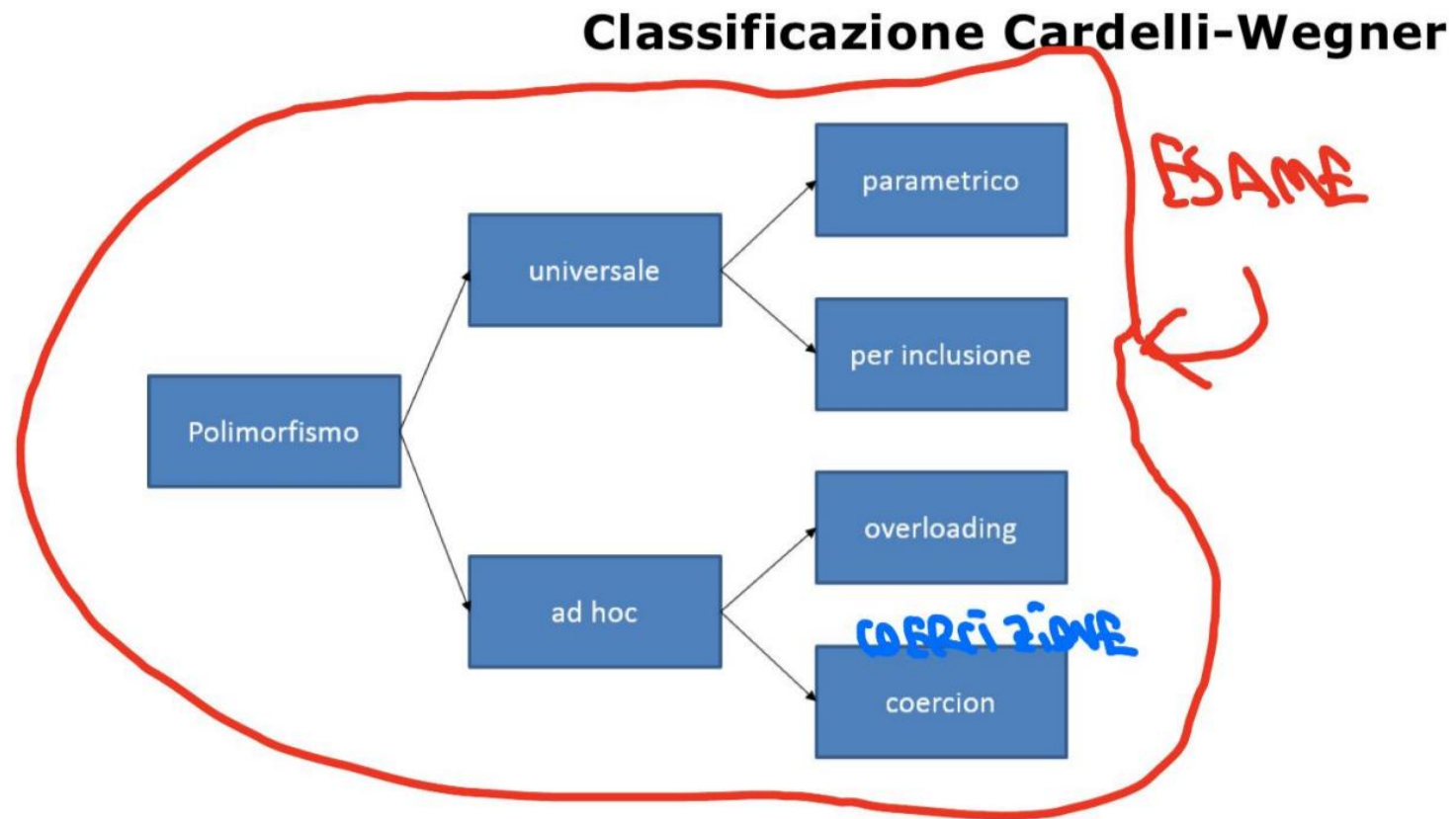
Cos'è l'ereditarietà di realizzazione (codice)?

Meccanismo di riutilizzo del codice.

Differenza fra signature e prototipo.

- La signature è composta da nome del metodo e lista degli argomenti con il loro tipo;
- Il prototipo è composto da signature e tipo restituito.

Classificazione Cardelli-Wegner.



Polimorfismo universale

È generale, applicabile a molti tipi.

Polimorfismo ad hoc?

È specifico, applicabile solo a tipi particolari.

Cos'è la coercizione?

Si ha quando una funzione converte in maniera implicita gli argomenti nel tipo applicabile.

```
double pot(double, double);
```

```
double x, y;
```

```
int n;
```

```
y = pot(x, n);
```

← ESAME

DATO QUESTO CODICE

Implicita conversione di tipo

DIRE SE SI TRATTA
DI COERCIZIONE O OVERLOADING

Coercizione, perché n, che è int, viene implicitamente convertito in double.

Cos'è il polimorfismo parametrico?

Si ha quando funzioni ed operatori vengono parametrizzati secondo il tipo a cui possono essere applicati.

Cos'è il polimorfismo per inclusione?

È un polimorfismo che si manifesta quando una classe derivata estende una classe base e, tramite l'ereditarietà, la classe derivata può essere trattata come se fosse un'istanza della classe base.

Differenza tra overriding e overloading.

- **Nell'overriding** si ridefinisce un metodo ereditato da una classe base mantenendo la stessa signature.
- **Nell'overloading** si definiscono più metodi con lo stesso nome, ma con signature diverse, nella stessa classe.

Esempio

file Base.java

```
class Base {  
    private int attr1;  
    int Metodo() {  
        return attr1;  
    }  
}
```

file Derivata.java

```
class Derivata extends Base {  
    private int attr2;  
    int Metodo() {  
        return super.Metodo() + attr2;  
    }  
}
```

ESEMPIO CHE FINISSE
IN ESAME

SE FOSSE PROTECTED
POTREI ANCORA
ACCEDERVI DA DENTRO
LA CLASSE? SÌ.

Compatibilità formale

L'istanza di una classe derivata è **formalmente compatibile** con il tipo della sua super classe *Base* `a = new Derivata()`;

Il tipo *Base* di *a* limita le operazioni di *Derivata*, se non fosse così viene generato un errore in compilazione

Esempio aggiungiamo alla classe *Derivata* un metodo `int Ciccia()`;

```
Base b1 = new Base(); ok  
Base b2 = new Derivata(); ok  
Derivata d = new Derivata(); ok  
b1.Metodo(); (OK)  
b2.Metodo(); (OK)  
b2.Ciccia(); (ERRORE)
```

ASSOCIATO AL CODICE DI
PRIMA-

Differenza tra upcasting e downcasting.

- L'**upcasting** consiste nel convertire (o assegnare) un oggetto di una classe derivata (sottoclasse) a un riferimento della sua classe base (superclasse).
- Il **downcasting** consiste nel convertire un riferimento di una superclasse a un tipo più specifico (sottoclasse).

Downcasting: esempio

file Base.java

```
class Base {  
    Private int attr1;  
    int Metodo() {  
        return attr1;}  
}
```

file Derivata.java

```
class Derivata extends Base {  
    Private int attr2;  
    int Metodo() {  
        return super.Metodo()+attr2;  
    }  
    int Ciccia() {  
        return attr2;}  
}
```

```
Base a = new Base();  
Base b = new Derivata();  
b.Ciccia(); //ERRORE
```

```
((Derivata)b).Ciccia(); // Downcast OK  
((Derivata)a).Ciccia(); // ERRORE
```

ESEMPIO CHE È
ALL'ESAME

Differenza tra sottoclasse e sottotipo.

- La sottoclasse non garantisce il principio di Liskov;
- Il sottotipo garantisce il principio di Liskov.

Cos'è la classe Object e che metodi pubblici definisce?

La classe Object è una superclasse, radice della gerarchia, che il compilatore usa se non ne viene espressamente usata un'altra.

Definisce i seguenti metodi pubblici:

- Public boolean equals(Object o) verifica l'uguaglianza tra due elementi;
- Public String toString() restituisce una rappresentazione stampabile di un oggetto;
- Public int hashCode() restituisce un valore intero legato al valore contenuto nell'oggetto.

Cos'è il garbage collection?

È un processo automatico gestito dalla JVM. Libera la memoria non allocata.

Che effetto ha il final su metodi, classi ed attributi?

- I metodi non possono essere ridefiniti da una sottoclasse;
- Le classi non possono avere sottoclassi;
- Gli attributi non possono essere modificati.

Come si impedisce il polimorfismo?

Con la keyword final.

Come si obbliga il polimorfismo?

Con la keyword abstract.

Come si rappresenta un dato astratto?

Tramite le classi astratte.

Differenza tra interfaccia e classe astratta.

- Un'interfaccia ha solo metodi astratti;
 - Una classe astratta ha anche metodi concreti.
-
- Le classi astratte possono estendere solo una classe (eventualmente astratta) per volta; Le interfacce possono estendere più interfacce in contemporanea;
 - Una classe astratta può essere ereditata da una classe (eventualmente astratta); Un'interfaccia può essere estesa solo da altre interfacce.

Quando conviene usare una classe astratta rispetto ad un'interfaccia?

Quando hai un comportamento comune da condividere tra le sottoclassi.

Che tipi di metodi possono avere interfacce e classi astratte?

- Classi astratte: protected/public;
- Interfacce: public.

Cosa significa static associato ad un metodo, classe, attributo?

- I metodi non sono legati ad un'istanza della classe, ma diventano comuni a tutte le istanze della classe;
- Gli attributi non sono legati ad un'istanza della classe, ma diventano comuni a tutte le istanze della classe;
- Le classi **non dipendono da un'istanza della classe esterna** e possono accedere solo ai metodi ed agli attributi static di quest'ultima.

Cosa s'intende per programmazione generica?

È un tipo di programmazione che crea costrutti in grado di accettare tipi di dati diversi.

Come si realizza la programmazione generica in Java?

Con il polimorfismo per inclusione o quello parametrico.

Cosa sono le collezioni?

Sono simili ad array ma di dimensione non fissa a priori.

Come funziona la gerarchia fra tipi parametrizzati?

Non esiste gerarchia.

Gerarchie tra tipi generici

Nemmeno *Sequence* di *Object* quindi è **supertipo** di *Sequence* di *A*

```
List<String> ls = new ArrayList<String>();
```

```
List<Object> lo = ls; //errore di tipo
```

```
ls.add(new Object()); //errore di tipo
```

CODICE
CHE
POTREBBE
CAPITARE
A ESAME.

QUESTO SOTTO NON
DOVREBBE

Cos'è la wildcard?

È un supertipo comune a tutti i tipi parametrizzati.

Cos'è $\langle ? \text{ extends } T \rangle$ e $\langle ? \text{ super } T \rangle$

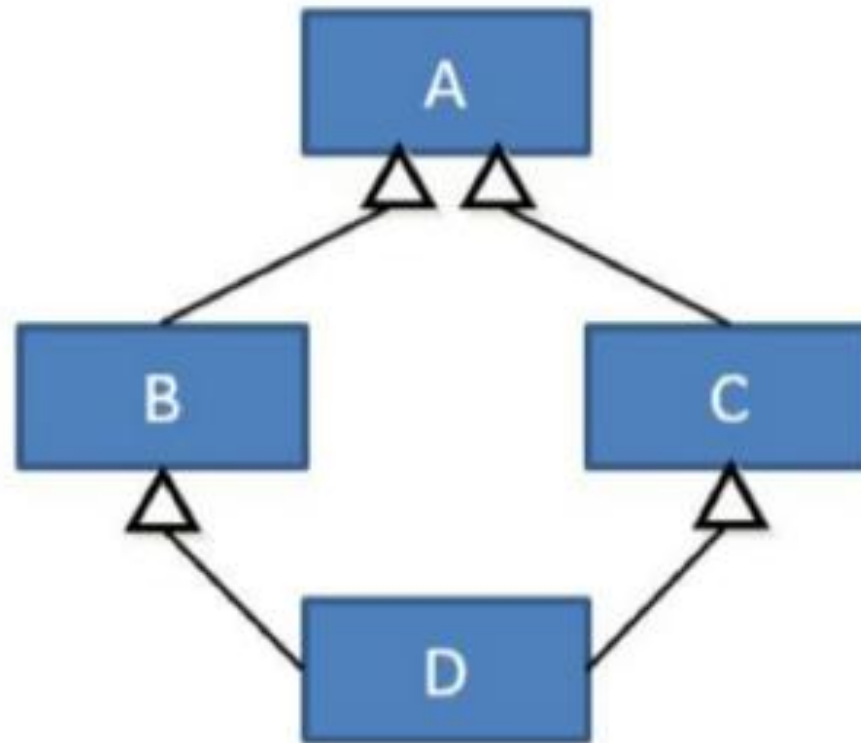
- $\langle ? \text{ extends } T \rangle$ può essere un qualunque sottotipo di T ;
- $\langle ? \text{ super } T \rangle$ può essere un qualunque supertipo del tipo T .

Cos'è l'ereditarietà multipla?

Si ha quando una classe eredita le proprie caratteristiche da più classi.

Cos'è il problema del diamante?

È l'ambiguità causata dall'ereditarietà multipla.



In Java è permessa l'ereditarietà multipla?

No.

Cosa sono i tipi enumerativi?

Sono classi speciali in cui i possibili valori del tipo sono stabiliti all'atto di definizione del tipo stesso.

Cosa sono le collection framework?

Sono classi contenitore.

Cosa sono i diagrammi delle classi?

Sono diagrammi che mostrano la struttura interna delle classi e le relazioni tra loro in un programma.

Cosa sono i diagrammi delle istanze?

È un diagramma che rappresenta come appaiono gli oggetti e le relazioni tra loro in uno specifico momento.

Cosa sono i diagrammi delle sequenze?

Sono diagrammi che mostrano come gli oggetti interagiscono tra loro lungo una sequenza temporale.

Che relazioni ci sono fra gli oggetti?

- Ereditarietà;
- Associazioni.

Che tipo di associazioni ci sono?

- Associazione tramite attributi delle classi;
- Associazione tramite classi specifiche.

Cos'è un pattern?

È una soluzione progettuale atta a risolvere un problema.

Cosa si intende per riusabilità?

Riusare stesse porzioni di codice per diversi problemi.

Come si applica concretamente la riusabilità?

- Librerie e componenti riusabili;
- Schemi trasformatzionali.

Cosa sono gli schemi trasformatazionali?

Sono metodi di sviluppo software che, a partire da una descrizione astratta, ne creano un'implementazione concreta.

Differenza tra incapsulamento ed information hiding?

- L'incapsulation è un raggruppamento in un entità unica di diversi elementi che vengono nascosti con lo scopo di proteggerli.
- L'information hiding consiste solo nel nascondere le implementazioni interne di una classe, mostrando solo ciò che è strettamente necessario per il suo uso.

Da cosa è costituito un design pattern?

- Nome;
- Descrizione del problema;
- Soluzione;
- Conseguenze.

Come si classificano i pattern?

- Strutturali;
- Creazionali;
- Comportamentali.

Cos'è il pattern Singleton?

Fa in modo che di una classe si possa creare una sola istanza, fornendo un punto d'accesso globale ad essa.

Cos'è il pattern Iterator?

Consente di iterare attraverso gli elementi di una collezione in modo sequenziale, senza rivelarne la struttura interna.

Cos'è il pattern Decorator?

Consente di aggiungere metodi a classi esistenti durante il run-time.

Cosa sono le eccezioni?

Sono errori che avvengono run-time.

Cosa si intende per affidabilità di un programma.

È la capacità di un programma di gestire gli errori.

Quali sono le cause principali di eccezioni?

- Mancanza di risorse;
- Parametri errati;
- Sequenza di operazioni illecita.

Cos'è il Try/Catch?

È un costrutto in cui si inserisce una porzione di codice che potrebbe causare errori.

Cos'è la classe throwable e quali sono le sue sottoclassi?

È una classe usata per gestire anomalie.

- Error per errori gravi del sistema non recuperabili;
- Exception per condizioni eccezionali gestibili dal programma.

Che metodi si usano per riportare l'anomalia?

- `String getMessage()` restituisce una stringa che descrive l'errore;
- `Void printStackTrace()` restituisce la posizione in cui si è verificato l'errore ed i metodi usati che lo hanno provocato.

Cosa sono i package in java?

Un **package** è un meccanismo per organizzare classi e interfacce correlate in gruppi logici.

Cos'è standard input?

È il flusso d'ingresso predefinito per le informazioni in ingresso al programma.

Cos'è standard output?

È il flusso d'uscita predefinito per i dati in uscita dal programma.

Cos'è standard error?

È il flusso d'errore predefinito per gli errori generati dal programma.

Cos'è la programmazione orientata agli aspetti?

- Permette di isolare comportamenti o funzionalità che sono **comuni a più moduli** del software (come log, sicurezza, gestione delle transazioni, gestione degli errori) e di **modularizzarli in un aspetto separato**.

Come funziona un compilatore?

- pre-processamento: vengono eliminati commenti, inclusi file, sostituite le macro;
- Analisi lessicale: verifica errori lessicali;
- Analisi sintattica: verifica errori sintattici;
- Analisi semantica;
- Generazione di codice intermedio ed ottimizzazione;
- Generazione di linguaggio macchina;
- Linking: crea l'eseguibile.

Cos'è il problema dell'arresto?

È l'impossibilità di sapere in anticipo quando un programma terminerà la sua esecuzione.

Cos'è un codice, nei linguaggi formali?

È un sottoinsieme di stringhe che non può essere interpretato in maniera ambigua.

Fasi della programmazione?

- Specifiche;
- Progettazione;
- Modellazione;
- Codifica;
- Verifica e correzione.